

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ
Кафедра Информационные системы
Дисциплина Программирование мобильных устройств

КУРСОВОЙ ПРОЕКТ (РАБОТА)

Тема Разработка мобильного приложения «Персонажи мультсериала Рик и Морти» на Flutter для Android

Выполнил студент _____ / Данилов В.В. /
подпись инициалы, фамилия

Курс 3 Группа Пибд-31

Направление/ специальность _____

Руководитель доцент, к.т.н, доцент
должность, ученая степень, ученое звание
Филиппов Алексей Александрович
фамилия, имя, отчество

Дата сдачи:
«___» _____ 20__ г.

Дата защиты:
«___» _____ 20__ г.

Оценка: _____

Студент _____ / Данилов В.В.
подпись инициалы, фамилия

ОТЗЫВ
руководителя на курсовой проект (работу)

студента _____ Данилова Владимира Владимировича _____

фамилия, имя и отчество

Факультет _____ ИСТ _____ группа _____ ПИБд-31 курс _____ 3 _____

Дисциплина _____ Программирование мобильных устройств _____

Тема проекта (работы) Разработка мобильного приложения «Персонажи
мультсериала Рик и Морти» на Flutter для Android _____

Отмечаются следующие моменты: актуальность темы исследования; соответствие содержания и структуры курсовой работы ее теме; степень разработанности проблемы, наиболее интересно исследованные вопросы. Оценивается степень самостоятельности и инициативы студента; умение пользоваться различными источниками информации; уровень его теоретической подготовки; умение анализировать научные материалы, делать практические выводы; знание основных концепций, научной и специальной литературы по избранной теме. Содержится оценка проекта (работы) руководителем.

Руководитель _____ доцент, к.т.н, доцент _____ / Филиппов А.А. /
должность, учёная степень, ученое звание подпись инициалы, фамилия

Введение

Современные технологии стремительно развиваются, и мобильные приложения стали неотъемлемой частью нашей повседневной жизни. Они не только упрощают доступ к информации, но и предоставляют удобные инструменты для развлечения, обучения и работы. В этом контексте разработка мобильных приложений является актуальной задачей, которая требует использования современных фреймворков и технологий для создания качественных и функциональных решений.

Одним из популярных фреймворков для разработки кроссплатформенных приложений является Flutter — инструмент, разработанный компанией Google. Flutter позволяет создавать приложения для различных платформ, включая Android, с использованием единого кода, что значительно упрощает процесс разработки и снижает временные затраты. Благодаря своей производительности, гибкости и обширной библиотеке виджетов, Flutter стал предпочтительным выбором для многих разработчиков.

В рамках данной курсовой работы предстоит разработать мобильное приложение для платформы Android, посвященное мультсериалу «Рик и Морти». Этот проект не только позволит закрепить теоретические знания и практические навыки работы с Flutter, но и создать удобное и интересное приложение для фанатов сериала. Приложение будет предоставлять информацию о персонажах мультфильма.

В данной работе представлено краткое описание предметной области, выделены основные сущности и бизнес-процессы, подлежащие автоматизации.

Были выделены следующие сущности:

- Персонаж

- Персонажи данного мультсериала, каждый из которых имеет имя, изображение, вид и статус.

- Сущность персонаж формирует основной набор данных, который отображается в приложении.
- Карточка персонажа
 - Графический элемент в интерфейсе приложения, который представляет собой сокращенную информацию о персонаже (изображение, имя, описание (вид и статус)).
 - Позволяет пользователю быстро ознакомиться с содержанием и выбрать интересующий объект для более детального изучения.
- Лайк
 - Механизм, который позволяет пользователю отмечать персонажей, которые ему понравились.
 - Информация о лайках сохраняется локально для дальнейшего доступа.

Используются следующие бизнес-процессы:

- Отображение каталога персонажей

Приложение должно автоматически запрашивать данные из API и выводить их в виде карточек. Каждая карточка содержит информацию о персонаже, который состоит из изображения, имени, вида и статуса. Это дает возможность пользователю быстро просматривать доступные варианты.

- Поиск и фильтрация данных

Пользователь может использовать поиск персонажей по имени для нахождения конкретных вариаций данного персонажа. Поисковые запросы отправляются через API, после чего приложение отображает результаты.

- Просмотр персонажа

При выборе карточки персонажа пользователь переходит на страницу с информацией о выбранном персонаже.

- Добавление в понравившееся

Для персонализации опыта пользователям предоставляется возможность ставить отметку "мне нравится". После установки лайка данные сохраняются в локальном хранилище, чтобы пользователь мог позже вернуться к своим предпочтениям.

- Локализация и настройка приложения:

Приложение поддерживает несколько языков, что делает его доступным для широкой аудитории. Также изменена стандартная иконка приложения для соответствия тематике.

- Обновление данных:

Пользователь может вручную обновлять список данных для получения актуальной информации.

Глава 1. Проектирование программной системы

1.1. Информационное обеспечение

В разрабатываемом приложении используются следующие сущности:

- HomeData
- CardDate

ER-диаграмма сущностей:

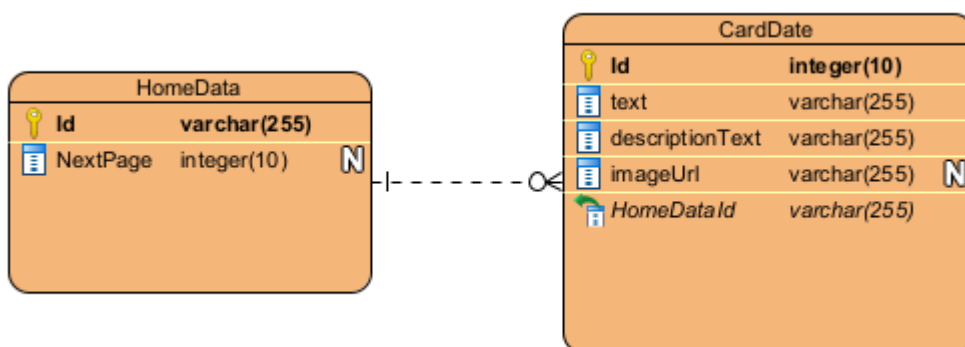


Рис. 1 ERD-диаграмма

Подробное описание каждой сущности:

1. CardDate

1.1 Id

1.1.1. Уникальный идентификатор

1.1.2. Число

1.2 Text

1.2.1. Наименование персонажа

1.2.2. Строка

1.3 DescriptionText

1.1.1. Описание персонажа (его вид и статус)

1.1.2. Строка

1.4 ImageUrl

1.4.1. Ссылка на изображение персонажа

1.4.2. Строка

1.5 .HomeDataId

1.6.1. Ссылка на HomeData

1.6.2. Строка

2. HomeData

2.1. Id

2.1.1. Уникальный идентификатор отображаемой страницы

2.1.2. Строка

2.2. NextPage

2.2.1. Номер следующей страницы

2.2.2. Число

Связь между сущностями:

HomeData – CardDate – Связь один ко многим. В одном HomeData может храниться множество элементов сущности CardDate.

1.2. Интерфейс пользователя

Интерфейс содержит в себе 2 различных экрана, содержащих различную информацию:

1. Главный экран

1.1. Экран, состоящий из следующих элементов:

1.1.1. Строка поиска

1.1.2. Кнопка, предназначенная для смены локализации

1.1.2.1. Выглядит как иконка флага, которая отображает выбранный язык

1.1.3. Список карточек персонажей. Каждая карточка состоит из следующих элементов:

1.1.3.1. Изображение

1.1.3.2. Наименование персонажа

1.1.3.3. Вид персонажа

1.1.3.4. Статус

1.1.3.5. Кнопка лайка

1.1.3.5.1. Первоначально выглядит как сердечко в виде с черной окантовкой и белой заливкой. При нажатии на лайк заливка и окантовка иконки становятся красными

2. Экран с подробной информацией о персонаже

2.1. Кнопка перехода на главный экран

2.2. Изображение

2.3. Наименование

2.4. Вид

2.5. Статус

Изображения экранов:

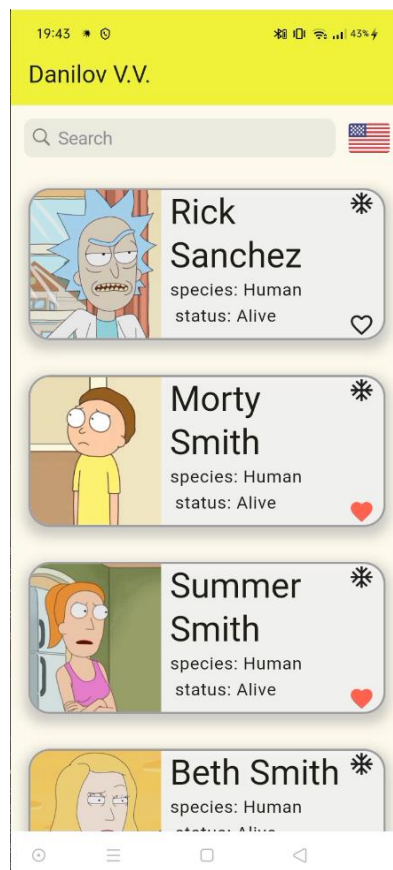


Рис 1. Главный экран с локализацией на английском языке

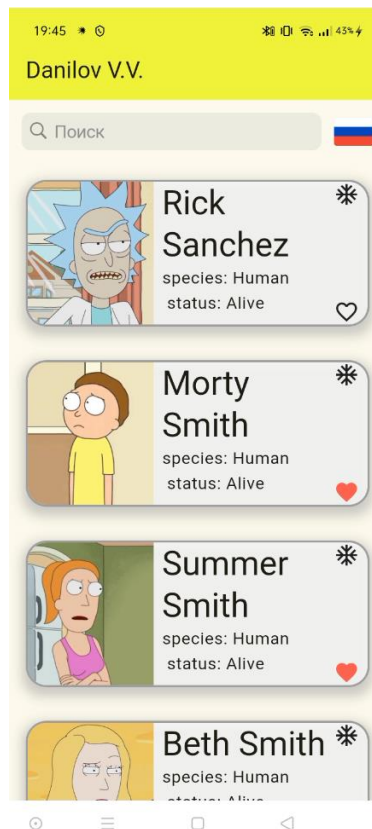


Рис 2. Главный экран с локализацией на русском языке

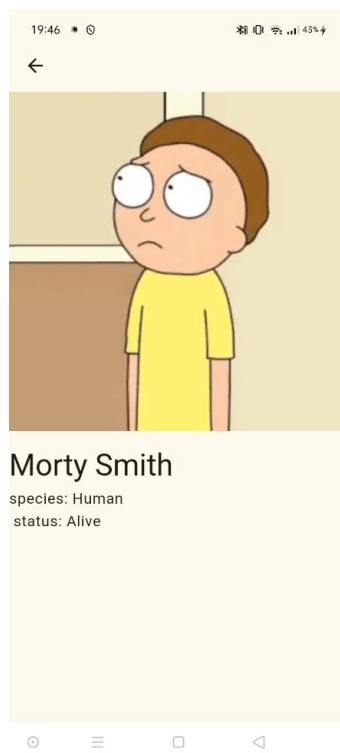


Рис 3. Экран с подробной информацией о произведении

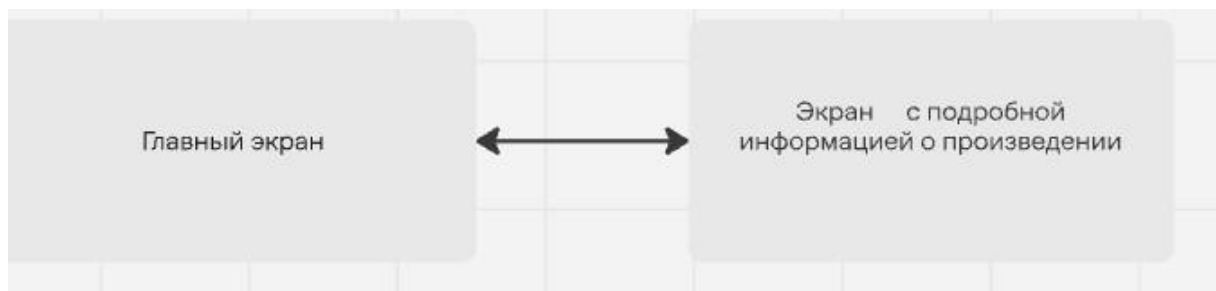


Рис 4. Карта экранов приложения

Пользователь может переходить с главного экрана на экран с подробной информацией. И наоборот, с экрана с подробной информацией переходить на главный экран.

1.3. Архитектура приложения

Приложение разработано с использованием многослойной архитектуры и паттерна BLoC (Business Logic Component), который отделяет бизнес-логику от пользовательского интерфейса. Это способствует упрощению управления состоянием, тестирования и дальнейшему развитию приложения. Особенности архитектуры приложения

1. Модульность:

Архитектура BLoC способствует созданию модульных компонентов, что позволяет легко переиспользовать логику в разных частях приложения или даже в разных проектах.

2. Многослойность

Каждый слой имеет свою определенную ответственность, что упрощает разработку и поддержку приложения. Например, слой представления отвечает только за отображение данных, а бизнес-логика обрабатывает все правила и операции.

3. Четкая организованность

BLoC обеспечивает ясную структуру кода, деля его на компоненты, такие как события (Events), состояния (States) и сам BLoC. Это улучшает читаемость и поддерживаемость кода.

4. Поддержка многопоточности

BLoC позволяет обрабатывать события и состояния в отдельных потоках, что может повысить производительность приложения.

Слои архитектуры и их функциональное назначение

1. Слой представления (UI Layer):

- Отвечает за отображение данных и взаимодействие с пользователем
- Отображает интерфейс (экраны, карточки, кнопки и т.д)
- Основные функции:
 1. Отображение списка карточек.
 2. Реакция на действия пользователя
 3. Навигация по карте экранов
- Взаимосвязь с другими слоями:
 1. Отправляет события в слой управления состоянием (BLoC).
 2. Получает состояния и данные от слоя управления состоянием для обновления интерфейса.

2. Слой управления состоянием (BLoC Layer):

- Содержит логику приложения: обработку пользовательских событий, преобразование данных и управление состоянием.
- Основной функционал:
 1. Запрос данных из слоя данных через репозитории.
 2. Фильтрация и преобразование данных.
 3. Управление лайками и обновлением состояния UI.
 4. Реализация Debounce для уменьшения количества запросов при поиске.

3. Слой доступа к данным (Data Layer):

- Отвечает за работу с внешними API и локальным хранилищем.

- Основной функционал:
 1. Выполнение запросов к API и получение данных о карточках персонажа.
 2. Сохранение локальных данных, таких как лайки, в SharedPreferences.
 3. Преобразование данных в объекты (DTO) для передачи в слой управления состоянием.
- Компоненты слоя данных:
 1. Data Sources:
 - Remote Data Source — для взаимодействия с внешними API.
 - Local Data Source — для работы с локальным хранилищем (SharedPreferences). Используется для сохранения понравившегося персонажа.
 2. Repositories:
 - Абстракция, объединяющая источники данных и предоставляющая интерфейсы для слоя бизнес-логики.

1.4. Программный интерфейс и функции серверной части

Приложение использует REST API для получения данных о списке персонажей. API предоставляет данные в формате JSON, которые затем преобразуются в объекты с помощью предоставленных DTO-классов.

Описание структуры и особенности REST API:

- Формат данных: JSON.
- Методы HTTP:
 - GET: используется для получения информации о списке персонажей с детальной информацией, а также о странице.
- Endpoints(пример если используется сторонний API):
 - / character: возвращает список персонажей с возможностью фильтрации и пагинации

Функциональные возможности серверной части:

1. Получение списка персонажей
 - Endpoints / character.
 - Поддерживаемые параметры запроса:
 - next: ссылка на следующую страницу.
 - q: строка для поиска персонажа по названию.
 - pages: количество элементов на странице.
2. Пагинация и фильтрация
 - API поддерживает пагинацию через параметры next и pages, а также поиск по названию через параметр q, что позволяет гибко управлять отображением данных в приложении.

Структура DTO:

DTO (Data Transfer Object) представляет собой объекты, используемые для получения и передачи данных между клиентом и сервером. В предоставленном коде определены следующие DTO:

1. CharactersDto
 - Описывает данные ответа, содержащего список персонажей.
 - Поля:
 - results: список персонажей (List< CharacterDataDto >).
 - info: данные о пагинации (PaginationDto).
2. CharacterDataDto
 - Описывает данные о каждом персонаже.
 - Поля:
 - id: идентификатор персонажа.
 - name: наименование персонажа.
 - status: статус.
 - species: вид персонажа.
 - image: изображение персонажа
3. PaginationDto
 - Описывает данные о пагинации.

○ Поля:

- count: длина страниц.
- pages: количество страниц.
- next: ссылка на следующую страницу.

Глава 2. Разработка программной системы

2.1 Слои приложения

Описание основных пакетов:

1. Основные зависимости для реализации функционала:

- `shared_preferences`: используется для локального хранения данных, таких как лайки.
- `dio` и `pretty_dio_logger`: для работы с REST API и логирования запросов.
- `json_annotation`, `json_serializable`, `build_runner`: для автоматической сериализации и десериализации данных (например, DTO).
- `flutter_bloc`, `equatable`: для управления состояниями и событийной логики.
- `intl` и `flutter_localizations`: для реализации локализации.

2. Иконки и стилизация:

- `flutter_launcher_icons`: замена стандартной иконки приложения на кастомную (`assets/icon.png`).

3. UI и графические элементы:

- `flutter_svg`: для работы с SVG-графикой, поддерживающей отображение изображений в формате SVG.

4. Навигация между слоями:

- Используются методы разделения интерфейсов (DTO, репозитории) и структуры BLoC для инкапсуляции логики и связывания UI с данными.

Взаимодействие между слоями:

1. Пользователь вводит запрос в поисковую строку.
2. UI отправляет событие в BLoC, которое обрабатывается бизнес-логикой.

3. BLoC вызывает репозиторий, который отправляет запрос через dio к REST API.
4. API возвращает данные в формате JSON.
5. DTO преобразует JSON в объект, который репозиторий возвращает в BLoC.
6. BLoC обновляет состояние, и данные передаются обратно в UI для отображения.

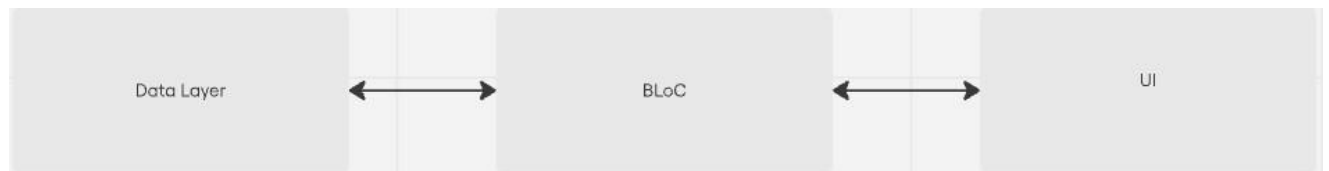


Рис 5. Схема взаимодействия между слоями

Основные классы и интерфейсы, работающие с хранилищем:

1. Интерфейс: ApiInterface

- определяет контракт для работы с API в приложении
- Методы:
 - `Future<HomeData?> loadData({OnErrorCallback? onError});`
Определяет метод для загрузки данных. Реализация этого метода обязана возвращать доменную модель HomeData.

2. Основной репозиторий: RickRepository

- Репозиторий реализует интерфейс ApiInterface и отвечает за получение данных из API.

Основные элементы:

- Поля:
 - `_dio`: объект Dio для отправки HTTP-запросов.
 - `_baseUrl`: базовый URL API (<https://rickandmortyapi.com>).
- Методы:
 - `loadData`:

1. Получает данные из API на основе переданных параметров и преобразует их в доменную модель HomeData.

- Параметры:

- onError: коллбек для обработки ошибок.
- q: строка поиска.
- page: номер страницы.
- pageSize: количество элементов на странице.

- Возвращаемое значение:

- Экземпляр HomeData или null в случае ошибки.

3. Типы данных и доменные модели:

- CardDate:

- Модель данных для отображения карточек персонажей:

1. text: название.
2. descriptionText: описание.
3. icon: иконка
4. imageUrl: ссылка на изображение.
5. id: уникальный идентификатор.

- HomeData:

- Модель для главной страницы:

1. data: список объектов CardDate.
2. nextPage: номер следующей страницы.

4. Мапперы (Extensions):

- Мапперы отвечают за преобразование DTO (Data Transfer Objects) в доменные модели (Domain Models). Они инкапсулируют логику преобразования и позволяют разделить данные, полученные из API, от внутренних структур приложения.

- Методы:

- CharacterDataDto.toDomain:

1. Преобразует объект CharacterDataDto в CardDate:

- CharactersDto.toDomain:

1. Преобразует объект CharactersDto в HomeData:

2.3 Слой для реализации бизнес-логики

Слой бизнес-логики (BLoC) отвечает за управление состоянием и реализацией бизнес-логики приложения. Он изолирует пользовательский интерфейс от взаимодействия с данными, что позволяет работать с репозиториями, обрабатывать информацию и передавать её в интерфейс.

Основные компоненты слоя бизнес-логики:

1. BLoC (Business Logic Component):

Этот компонент управляет событиями и состояниями приложения.

Наследуются от Bloc<Event, State>.

2. Events (События):

События представляют собой действия пользователя или изменения, которые необходимо обработать. .

3. States (Состояния):

Состояния определяют текущий статус приложения, который передается в пользовательский интерфейс. Наследуются от Equatable.

Содержат get props, необходимый для сравнения объектов.

Основные «блоки»:

1. HomeBloc:

Класс, который отвечает за отображение главной страницы приложения

1.1. HomeBloc:

1.1.1. Методы:

□ _onLoadData(HomeLoadDataEvent event, Emitter<HomeState> emit)

- Обработывает событие HomeLoadDataEvent, загружая данные из репозитория.

1.1.2.Поля:

- `rep` ○ Хранит `RickRepository`

1.2. HomeEvent

1.2.1. HomeLoadDataEvent

- Поля: ○

`Search`

- Строка запроса для поиска по названию ○

`nextPage`

- Номер следующей страницы для пагинации

1.3. HomeState

1.3.1.Методы

- `HomeState.copyWith({HomeData? data, bool? isLoading, bool? isPaginationLoading, String? error})` ○ Обновляет необходимые поля

1.3.2.Поля

- `Data`

- Хранит в себе `HomeData`

- `isLoading` ○ Флаг, показывающий грузятся ли сейчас данные из API

2. LikeBloc:

2.1. LikeBloc:

2.1.1.Методы

- `_onLoadLikes(LoadLikesEvent event, Emitter<LikeState> emit)`
 - Обработывает событие `LoadLikesEvent`, подгружая из `SharedPreferences` информацию о том, какие персонажи были отмечены как понравившиеся.

- `_onChangeLike(ChangeLikeEvent event, Emitter<LikeState> emit)` ○ Обработывает событие `ChangeLikeEvent`, которое ставит/убирает лайк с персонажа.

2.2. LikeEvent

Все события наследованы от абстрактного класса `LikeEvent`

2.2.1. LoadLikesEvent

- Методы и поля отсутствуют

2.2.2. ChangeLikeEvent

- Поля ○ `Id` Идентификатор персонажа, которому нужно поставить/убрать лайк.

2.3. LikeState

2.3.1. Поля:

- `List<String>? likedIds` ○ Список идентификаторов персонажей, которая понравилась

3. LocaleBloc

3.1. LocaleBloc

3.1.1. Метод:

- `_onChangeLocale(ChangeLocaleEvent event, Emitter<LocaleState> emit)` ○ Обработывает событие `ChangeLocaleEvent`, заменяющий текущую локализацию на следующую из существующих. (В приложении присутствует английская и русская)

3.2. LocaleEvent

3.2.1. ChangeLocaleEvent

- Не содержит методов и полей

3.3. LocaleState

3.3.1. Поля

- `Locale currentLocale`

- Текущая локализация

2.4 Слой представления

1. Основные классы и интерфейсы:

1.1. _Card (StatelessWidget) 1.1.1.Элемент списка карточек

1.1.2.Поля:

- Text ○ Наименование персонажа ○ Строка
- DescriptionText ○ Описание персонажа ○ Строка
- ImageUrl ○ Ссылка на изображение ○ Строка
- onLike ○ Обработчик лайков ○ Callback
- onTap
 - Обработчик нажатия (открытия карточки)
 - Callback
- Id
 - Идентификатор персонажа ○ Строка
- IsLiked ○ Статус лайка ○ Bool

1.1.3.Методы:

- build(BuildContext context) ○ Создает визуальное отображение карточки.

1.2. SvgObjects

1.2.1.Класс, кэширующий SVG-изображения

□ Метод:

- Init()
 - Загружает SVG в кэш.

1.3. SvgRu и SvgUK

1.3.1.Наследники SvgObjects

- Метод:

- build(BuildContext context)

➤ Создает визуальное отображение конкретного SVG изображения

1.4. Body (StatefulWidget)

1.4. 1.Класс отображения основной части главного экрана.

1.5. _BodyState

1.5. 1.Класс отображающий основную часть главного экрана. Является наследником от State(Body)

1.5.2.Поля:

- searchController ○ Контроллер, ответственный за изменение текста в строке поиска.
- scrollController ○ Контроллер, ответственный за пролистывание карточек на главном экране

1.5.3.Методы:

- initState() ○ Инициализирует SVG изображения ○ Загружает данные для первой загрузки.
 - Добавляет контроллеру метод, который он будет вызывать
- _onNextPageListener() ○ Подгружает новые карточки при пролистывании до конца списка карточек.
- _dispose ○ Очищает ресурсы при удалении данного экрана из дерева виджетов.
- _navToDetails(BuildContext context, CardDate data) ○ Открывает карточку, на которую нажали
- _onLike(String? id, String title, bool isLiked) ○ Вызывает событие ChangeLikeEvent
- _showLiked(BuildContext context, String title, bool isLiked)

- Показывает уведомление снизу (Снекбар) о поставленном/убранном лайке.
- `_onRefresh()` ○ Обновляет страницу

1.6. MyHomePage (StatefulWidget)

1.6. 1.Класс отображения главной страницы

1.7. `_MyHomePageState`

1.7. 1.Основной класс отображения главной страницы. Является наследником `State<MyHomePage>`

1.7.2.Методы:

- `InitState()` ○ Создает при первом запуске приветственное сообщение
- `build(BuildContext context)` ○ Создает отображение главной страницы и вызывает `Body`

1.8. MyApp (StatelessWidget)

1.8.1.Главный класс для отображения главной страницы

1.8.2.Метод:

- `build(BuildContext context)` ○ Создает главную страницу, вызывая `MyHomePage`

Глава 3. Руководство пользователя

Приложение «Персонажи мультсериала Рик и Морти» представляет собой инструмент для любителей данного мультсериала, который предназначен для быстрого поиска и просмотра информации о нужном персонаже.

Основные возможности приложения:

- Поиск персонажа по названию : помогает быстро найти определенного персонажа.
- Просмотр информации о персонаже: позволяет узнать ключевую информацию о нужном персонаже.
- Детальная информация о персонаже: позволяет более подробно узнать о персонаже.
- Смена языка интерфейса: позволяет использовать пользователям как русским, так и иностранным.
- Автоматическая подгрузка данных: удобный способ просмотра новых персонажей по мере прокрутки списка.

Приложение разработано с простым и интуитивно понятным интерфейсом, что делает его доступным для пользователей любого уровня. Каталог персонажей — надежный инструмент, который поможет найти новую информацию, вспомнить старых героев и отслеживать новых.

Целевая аудитория приложения:

- Фанаты мультсериала.
- Новички, желающие начать знакомство с данным произведением.
- Молодежь и подростки.

Процесс запуска приложения «Каталог персонажей» максимально прост и не требует специальных навыков. Приложение разработано для мобильных устройств, что делает его доступным в любое время и в любом месте.

Для начала работы с приложением требуется выполнить следующие шаги:

1. Найти приложение на устройстве:

- После установки приложения на мобильное устройство его иконка появится на главном экране или в списке установленных программ.
- Иконка Каталога персонажей имеет узнаваемый дизайн в виде енота, который, позволяет легко находить иконку приложения даже при большом количестве установленных программ.

2. Запустить приложение:

- Нажать на иконку приложения одним касанием экрана.
- При запуске приложения происходит кратковременная загрузка данных, после чего автоматически открывается главный экран приложения.

3. Первый запуск приложения:

- При запуске приложение автоматически подгрузит актуальные данные и сформирует список карточек персонажей на главном экране приложения.
- Для корректной работы рекомендуется обеспечить доступ к интернету, чтобы приложение могло загружать и обновлять информацию.

После успешного запуска Каталога персонажей пользователь сразу попадает на главный экран, где представлены карточки персонажей и функциональные элементы интерфейса. Главный экран разработан таким образом, чтобы пользователь мог интуитивно начать работу с приложением — будь то поиск, просмотр карточек или смена языка.

Примечание: При отсутствии подключения к интернету приложение может отображать уведомление о необходимости подключения для корректной загрузки данных.

Таким образом, запуск приложения занимает минимальное количество времени и позволяет пользователю быстро приступить к использованию его основных функций.

Главный экран является центральной частью приложения Каталог персонажей и предоставляет пользователю доступ ко всем ключевым функциям.

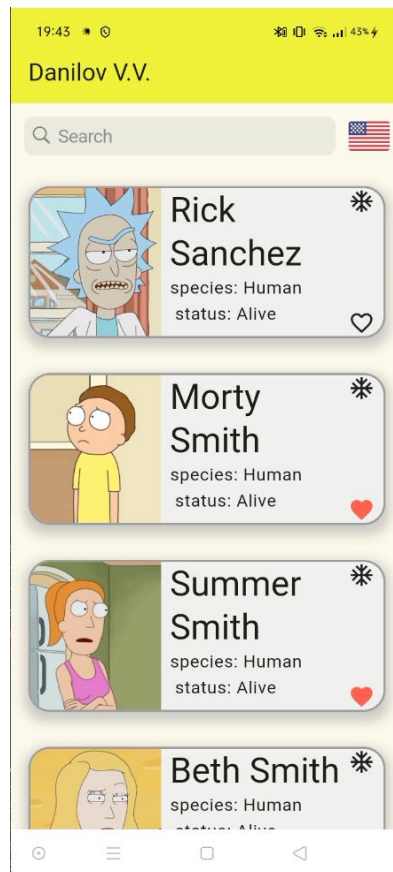


Рис. 6 Главный экран приложения

Элементы главного экрана:

1. Поисковая строка

- Используется для ввода названия для нахождения определенного персонажа.
- Ввод текста активирует фильтрацию карточек, отображаются только те герои, что соответствуют введенному запросу.
- Пример:

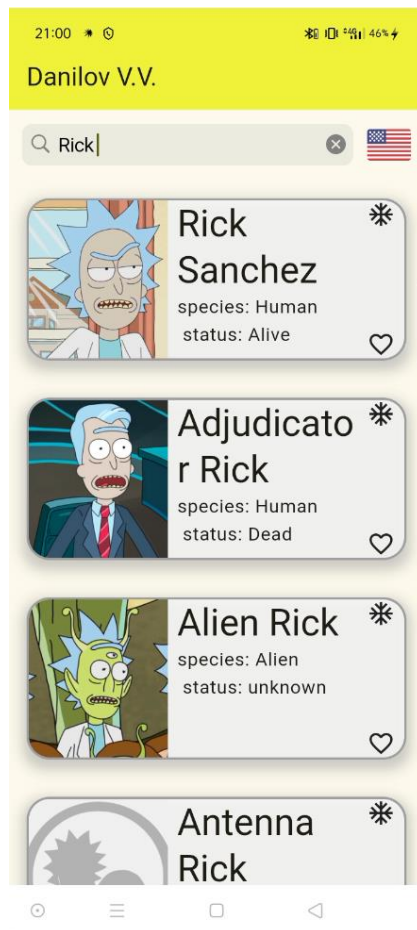


Рис 7. Запрос «Rick»

2. Кнопка смены локализации:

- Иконка флага справа от поисковой строки, отображается флаг страны, язык которой используется
- При нажатии язык интерфейса изменяется на русский или английский
- Пример:
Если язык интерфейса был английским, то при нажатии он изменяется на русский

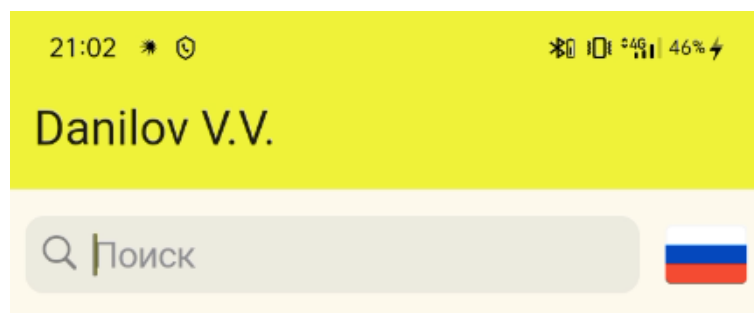


Рис. 8 Кнопка смены локализации

3. Список карточек персонажей

- Основная область экрана, расположенная под поиском, в которой содержатся карточки с изображениями, описанием.
- Прокрутка вниз поддерживает динамическую подгрузку, когда пользователь доходит до конца списка, новые карточки подгружаются спустя короткое время
- Элементы карточки:
 - Изображение — изображение персонажа
 - Описание — краткая информация о герое
 - Кнопка лайка — позволяет добавлять персонажа в понравившегося или удалять из понравившихся
- Пример:

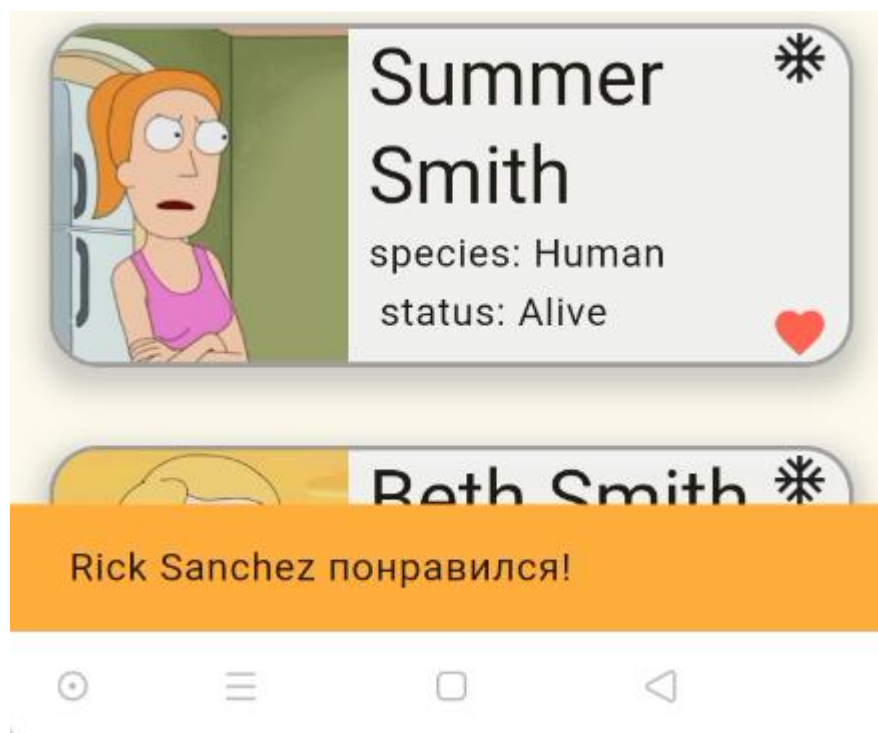


Рис. 9 Добавление в понравившееся

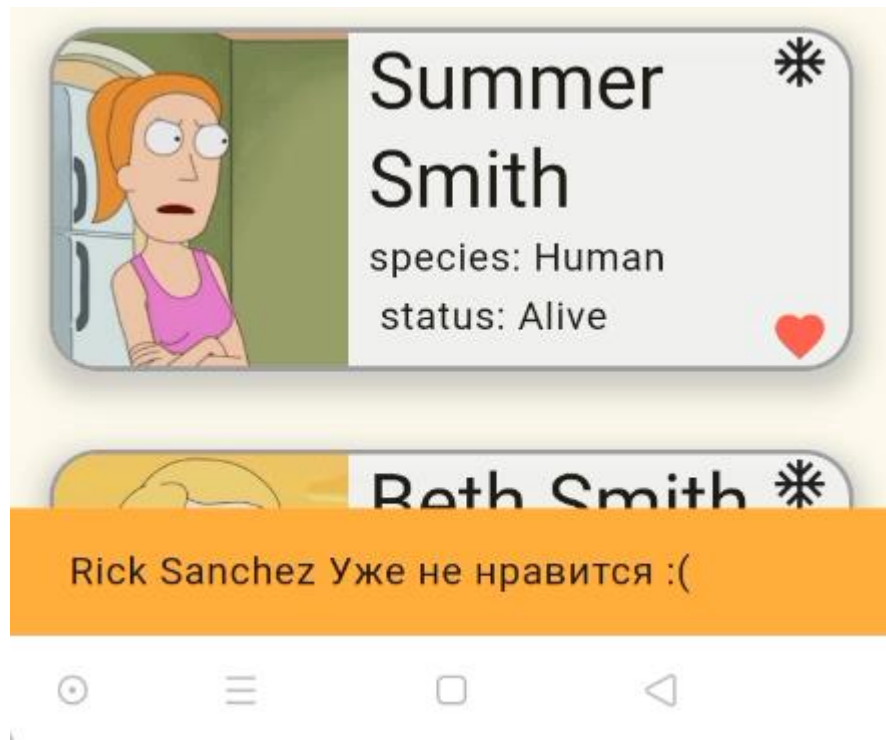


Рис. 10 Удаление из понравившихся

4. Индикатор загрузки

- Находится внизу экрана под последней карточкой на странице, выглядит как круговой индикатор загрузки при подгрузке новых данных
- Пример:

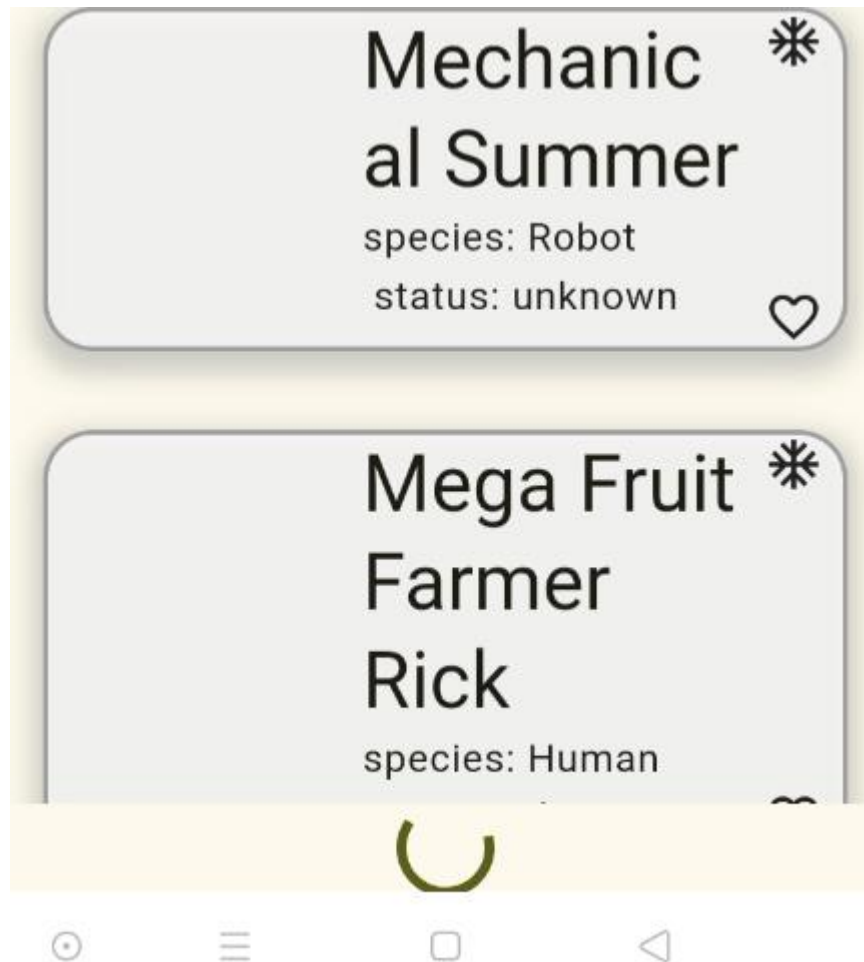


Рис. 11 Индикатор загрузки

Экран дополнительной информации открывается при нажатии на карточку персонажа на главном экране. Этот экран может предоставлять более детальные сведения о выбранном герое.

Элементы экрана дополнительной информации:

1. Кнопка «Назад»

- Расположена в самом верху слева, выглядит как стрелка налево
- При нажатии возвращает на главный экран приложения •

Пример:

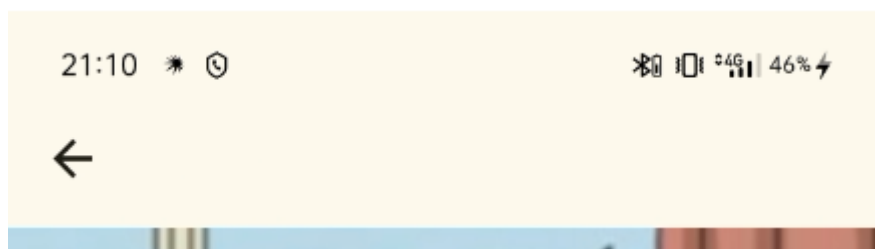


Рис. 12 Кнопка «Назад»

2. Обложка персонажа

- Отображается под кнопкой «Назад»
 - Помогает пользователю визуально идентифицировать героя
3. Наименование
- Имя героя выделено крупным шрифтом
4. Вид и статус персонажа
- Вид позволяет узнать пользователю кем является данный герой.
 - Статус позволяет пользователю узнать, что на последний момент выхода мультсериала произошло с героем
 - Пример:



Рис. 13 Экран с подробной информацией

Контрольный пример работы с приложением

Шаг 1: Запуск приложения

- Пользователь находит иконку Каталога персонажей на рабочем столе мобильного устройства.
- Пользователь запускает приложение одним нажатием.

Шаг 2: Использование поиска

- Пользователь вводит названия интересующего героя в строку поиска (например, "Rick").
- Приложение отображает все карточки, соответствующие введённому запросу.

Шаг 3: Просмотр карточки

- Пролистав список карточек, пользователь выбирает интересующую.
- Нажатие на карточку переводит пользователя к экрану дополнительной информации.

Шаг 4: Просмотр детальной информации

- На экране дополнительной информации пользователь ознакомливается с изображением, описанием и т.д..

Шаг 5: Смена языка

- Пользователь возвращается на главный экран.
- Пользователи нажимает на иконку с изображением флага для смены языка интерфейса.

Рекомендации по использованию

Для того чтобы работа с приложением Каталог персонажей была максимально продуктивной, удобной и приятной, нужно следовать следующим рекомендациям:

1. Требуется стабильное подключение к интернету
 - Приложение использует интернет-соединение для загрузки карточек персонажей и дополнительной информации. Стабильный интернет гарантирует быструю работу поиска и плавную прокрутку карточек.
 - Рекомендуется использовать Wi-Fi соединение для экономии мобильного трафика, особенно при активном использовании приложения.

2. Использование строки поиска для ускоренного доступа
 - При большом количестве карточек персонажей, ручной поиск может занять время.
 - Чтобы быстрее найти нужного героя, можно использовать поисковую строку для ввода названия. Чем точнее запрос, тем быстрее будет результат.
3. Смена локализации для удобства
 - Использование функции смены языка интерфейса в зависимости от предпочтений пользователя. С помощью одного нажатия пользователь может выбрать более удобный для него язык.
 - Это особенно полезно для пользователей, которые ищут персонажа на английском и ищут его оригинальное название.
4. Изучение карточек
 - Каждая карточка персонажа содержит базовую информацию: изображение, наименование, краткое описание. При выборе нового героя для изучения рекомендуется просмотреть все эти элементы.
 - Нажатие на карточку для получения дополнительной информации, где может быть представлено расширенное описание.
5. Использование прокрутки для просмотра новых карточек
 - Приложение поддерживает бесконечную прокрутку, что позволяет подгружать новые карточки персонажей по мере прокрутки вниз, до тех пор, пока список персонажей с фильтрацией или без не закончится.

Выводы

Приложение "Каталог персонажей " представляет собой мощный инструмент для всех любителей мультсериала, предлагая удобный доступ к обширной библиотеке героев. Благодаря интуитивно понятному интерфейсу и функциональным возможностям, пользователи могут легко находить, фильтровать и наслаждаться своими любимыми персонажами в любое время и в любом месте.

- В ходе работы был проведен анализ предметной области и определены требования к разработке приложения.
- Были созданы интерфейсы, которые обеспечивают интуитивную навигацию по приложению
- В ходе реализации приложения была использована современная платформа Flutter, позволяющая разрабатывать кроссплатформенные решения для Android и iOS.
- Также в ходе работы стало тестирование приложения на реальных устройствах. В результате тестирования было выявлено и устранено несколько незначительных ошибок, что позволило повысить производительность и стабильность приложения. Приложение корректно отображает данные, поддерживает ввод запросов в строку поиска и успешно переключает локализацию интерфейса.
- Разработанное приложение отвечает современным требованиям к мобильным решениям и обеспечивает:
 - Удобство взаимодействия: интуитивно понятный интерфейс и простая структура приложения;
 - Функциональность: быстрый поиск по базе данных персонажей и детальное отображение информации;
 - Доступность: поддержка двух языков, что делает приложение удобным для международной аудитории;
 - Производительность: минимальные задержки при работе даже на устройствах со средними техническими характеристиками.

Можно сделать вывод, что были выполнены все поставленные при разработке задачи.

Список литературы

1. Данилов В.В. [Электронный ресурс] // Гит УлГТУ : [сайт]. — URL: https://git.is.ulstu.ru/VoOneChek/PIbd_31_Danilov_V_V_PMU.git
2. Савельев Я. К. Flutter с нуля [Электронный ресурс] // Ютуб : [сайт]. — URL: https://youtube.com/playlist?list=PLEcfA4KuxR5bGM5LCzcBDkOrOX0dBZ4HN&si=0QUax8D_-GNiZCip
3. Введение в дарт. / [Электронный ресурс] // : [сайт]. — URL: <https://metanit.com/dart/tutorial/1.1.php>
4. Руководство по фреймворку Flutter / [Электронный ресурс] // : [сайт]. — URL: <https://metanit.com/dart/flutter/>
5. Онлайн компилятор Dart / [Электронный ресурс] // DartPad : [сайт]. — URL: <https://dartpad.dev/>

Исходный код

```
Main.dart
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import
'package:flutter_labs_app/data/repositories/rick_reposi
tory.dart';
import
'package:flutter_labs_app/presentation/home_page/bloc/b
loc.dart';
import
'package:flutter_labs_app/presentation/home_page/home_p
age.dart';
import
'package:flutter_labs_app/presentation/like_bloc/like_b
loc.dart';
import
'package:flutter_labs_app/presentation/locale_bloc/loca
le_bloc.dart';
import
'package:flutter_labs_app/presentation/locale_bloc/loca
le_state.dart';

import 'components/locale/110n/app_locale.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider<LocaleBloc>(
      lazy: false,
      create: (context) =>
        LocaleBloc(Locale(Platform.localeName)),
      child: BlocBuilder<LocaleBloc, LocaleState>(
        builder: (context, state) {
          return MaterialApp(
            title: 'Flutter Demo',
            locale: state.currentLocale,
            localizationsDelegates:
              AppLocale.localizationsDelegates,
            supportedLocales:
              AppLocale.supportedLocales,
            debugShowCheckedModeBanner: false,
            theme: ThemeData(
              colorScheme:
                ColorScheme.fromSeed(seedColor: Colors.limeAccent),
              useMaterial3: true,
            ),
            home: RepositoryProvider<RickRepository>(
              lazy: true,
              create: (_) => RickRepository(),
              child: BlocProvider<LikeBloc>(
                lazy: false,
                create: (context) => LikeBloc(),
                child: BlocProvider<HomeBloc>(
                  lazy: false,
                  create: (context) =>
                    HomeBloc(context.read<RickRepository>()),
                  child: const MyHomePage(title:
                    'Danilov V.V.'),
                ),
              ),
            ),
          );
        },
      ),
    );
  }
}
```

```

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../components/locale/lib/app_locale.dart';
import 'locale_events.dart';
import 'locale_state.dart';

class LocaleBloc extends Bloc<LocaleEvent, LocaleState> {
  LocaleBloc(Locale defaultLocale) :
    super(LocaleState(currentLocale: defaultLocale)) {
    on<ChangeLocaleEvent>(_onChangeLocale);
  }

  Future<void> _onChangeLocale(ChangeLocaleEvent event,
    Emitter<LocaleState> emit) async {
    final toChange = AppLocale.supportedLocales
      .firstWhere((e) => e.languageCode !=
state.currentLocale.languageCode);
    emit(state.copyWith(currentLocale: toChange));
  }
}

abstract class LocaleEvent {
  const LocaleEvent();
}

class ChangeLocaleEvent extends LocaleEvent {
  const ChangeLocaleEvent();
}

import 'dart:ui';
import 'package:equatable/equatable.dart';
import
'package:copy_with_extension/copy_with_extension.dart';

part 'locale_state.g.dart';

@CopyWith()
class LocaleState extends Equatable {
  final Locale currentLocale;

  const LocaleState({required this.currentLocale});

  List<Object?> get props => [currentLocale];
}

import 'package:flutter_bloc/flutter_bloc.dart';
import
'package:shared_preferences/shared_preferences.dart';
import 'like_event.dart';
import 'like_state.dart';

const String _likedPrefsKey = 'liked';

class LikeBloc extends Bloc<LikeEvent, LikeState> {
  LikeBloc() : super(const LikeState(likedIds: [])) {
    on<ChangeLikeEvent>(_onChangeLike);
    on<LoadLikesEvent>(_onLoadLikes);
  }

  Future<void> _onLoadLikes(LoadLikesEvent event,
    Emitter<LikeState> emit) async {
    final prefs = await
SharedPreferences.getInstance();
    final data = prefs.getStringList(_likedPrefsKey);

    emit(state.copyWith(likedIds: data));
  }

  Future<void> _onChangeLike(ChangeLikeEvent event,
    Emitter<LikeState> emit) async {

```

```

    final updatedList =
List<String>.from(state.likedIds ?? []);

    if (updatedList.contains(event.id)) {
        updatedList.remove(event.id);
    } else {
        updatedList.add(event.id);
    }

    final prefs = await
SharedPreferences.getInstance();

    prefs.setStringList(_likedPrefsKey, updatedList);

    emit(state.copyWith(likedIds: updatedList));
}
}

```

```

abstract class LikeEvent {

    const LikeEvent();

}

```

```

class LoadLikesEvent extends LikeEvent {

    const LoadLikesEvent();

}

```

```

class ChangeLikeEvent extends LikeEvent {

    final String id;

    const ChangeLikeEvent(this.id);

}

```

```

import 'package:equatable/equatable.dart';

import

'package:copy_with_extension/copy_with_extension.
dart';

```

```

part 'like_state.g.dart';

```

```

@CopyWith()

```

```

class LikeState extends Equatable {

    final List<String>? likedIds;

```

```

    const LikeState({required this.likedIds});

```

```

    @override

```

```

    List<Object?> get props => [likedIds];

}

```

```

import 'package:flutter/cupertino.dart';

```

```

import 'package:flutter/material.dart';

```

```

import 'package:flutter_bloc/flutter_bloc.dart';

```

```

import

```

```

'package:flutter_labs_app/components/extensions/co
ntext_x.dart';

```

```

import

```

```

'package:flutter_labs_app/data/repositories/mock_re
pository.dart';

```



```

import                                                    State<MyHomePage> createState() =>

'package:flutter_labs_app/data/repositories/rick_rep _MyHomePageState();

ository.dart';                                           }

import

'package:flutter_labs_app/presentation/details_page/ class _MyHomePageState extends

details_page.dart';                                     State<MyHomePage> {

                                                         final Color _color = Colors.limeAccent;

import '../components/utils/debounce.dart';

import '../domain/models/card.dart';                     @override

import '../common/svg_objects.dart';                     Widget build(BuildContext context) {

import '../like_bloc/like_bloc.dart';                     return Scaffold(

import '../like_bloc/like_event.dart';                     appBar: AppBar(

import '../like_bloc/like_state.dart';                     backgroundColor: _color,

import '../locale_bloc/locale_bloc.dart';                 title: Text(widget.title),

import '../locale_bloc/locale_events.dart';                ),

import '../locale_bloc/locale_state.dart';                 body: const Body(),

import 'bloc/bloc.dart';                                   );

import 'bloc/events.dart';                                 }

import 'bloc/state.dart';                                 }

part 'card.dart';

                                                         class Body extends StatefulWidget {

class MyHomePage extends StatefulWidget {                 const Body({super.key});

  const MyHomePage({super.key, required this.title});

  final String title;                                     @override

                                                         State<Body> createState() => _BodyState();

  @override                                              }

```

```

class _BodyState extends State<Body> {
    final scrollController = ScrollController();
    final searchController = TextEditingController();

    @override
    void initState() {
        SvgObjects.init();

        WidgetsBinding.instance.addPostFrameCallback((_) {
            context.read<HomeBloc>().add(const
HomeLoadDataEvent());
            context.read<LikeBloc>().add(const
LoadLikesEvent());
        });
        scrollController.addListener(_onNextPageListener);

        super.initState();
    }

    void _onNextPageListener() {
        if (scrollController.position.atEdge) {
            final bloc = context.read<HomeBloc>();
            if (!bloc.state.isPaginationLoading) {
                bloc.add(
                    HomeLoadDataEvent(search:
                    searchController.text, nextPage:
                    bloc.state.data?.nextPage));
            }
        }

        @override
        void dispose() {
            searchController.dispose();
            scrollController.dispose();
            super.dispose();
        }

        final RickRepository repo = RickRepository();
        var data = RickRepository().loadData();

        void _showSnackBar(BuildContext context, String
        title, bool isLiked) {
            WidgetsBinding.instance.addPostFrameCallback((_) {
                ScaffoldMessenger.of(context).showSnackBar(SnackB
                ar(
                    content: Text(
                        'Title ${isLiked ? context.locale.liked :
                        context.locale.disliked}',

```

```

        style: Theme.of(context).textTheme.bodyLarge,
      ),
      backgroundColor: Colors.orangeAccent,
      duration: const Duration(seconds: 1),
    ));
  });
}

void _navToDetails(BuildContext context, CardDate
data) {
  Navigator.push(context,
    CupertinoPageRoute(builder: (context) =>
      DetailsPage(data)));
}

Future<void> _onRefresh() {
  context.read<HomeBloc>().add(HomeLoadDataEvent(
    search: searchController.text));
  return Future.value(null);
}

@override
Widget build(BuildContext context) {
  return Padding(
    padding: EdgeInsets.only(top:
      MediaQuery.of(context).padding.top),
    child: Column(
      children: [
        Row(
          children: [
            Expanded(
              flex: 4,
              child: Padding(
                padding: const EdgeInsets.all(12),
                child: CupertinoSearchTextField(
                  controller: searchController,
                  placeholder: context.locale.search,
                  onChanged: (search) {
                    Debounce.run(
                      () =>
                        context.read<HomeBloc>().add(HomeLoadDataEvent(
                          search: search)));
                  )),
                ),
            GestureDetector(
              onTap: () =>
                context.read<LocaleBloc>().add(const
                  ChangeLocaleEvent()),
              child: SizedBox.square(
                dimension: 50,
                child: Padding(
                  padding: const EdgeInsets.only(right: 12),

```

```

        child: BlocBuilder<LocaleBloc,
LocaleState>(
          builder: (context, state) {
            return
state.currentLocale.languageCode == 'ru'
              ? const SvgRu()
              : const SvgUk();
            },
          ),
        ),
      ],
    ),
    BlocBuilder<HomeBloc, HomeState>(
      builder: (context, state) => state.error != null
        ? Text(
            state.error ?? "",
            style:
Theme.of(context).textTheme.headlineSmall?.copyWi
th(color: Colors.red),
          )
        : state.isLoading
          ? CircularProgressIndicator()
          : BlocBuilder<LikeBloc, LikeState>(
            builder: (context, likeState) =>
Expanded(
          child: RefreshIndicator(
            onRefresh: _onRefresh,
            child: ListView.builder(
              controller: scrollController,
              padding: EdgeInsets.zero,
              itemCount:
state.data?.data?.length ?? 0,
              itemBuilder: (context, index) {
                final data =
state.data?.data?[index];
                return data != null
                  ? _Card.fromData(
                      data,
                      onLike: _onLike,
                      isLiked:
likeState.likedIds?.contains(data.id) == true,
                      onTap: () =>
_navToDetails(context, data),
                    )
                  : const SizedBox.shrink();
              },
            ),
          ),
        ),
      ),
    ),
  ),
)

```

```

    ),

    BlocBuilder<HomeBloc, HomeState>{

      builder: (context, state) =>

state.isPaginationLoading

        ? const CircularProgressIndicator()

        : const SizedBox.shrink())

    ],

  ));

}

void _onLike(String? id, String title, bool isLiked) {

  print("$id $title, $isLiked");

  if (id != null) {

context.read<LikeBloc>().add(ChangeLikeEvent(id));

    _showSnackBar(context, title, !isLiked);

  }

}

}

part of 'home_page.dart';

typedef OnLikeCallback = void Function(String? id,

String title, bool isLiked)?;

class _Card extends StatelessWidget {

  final String text;

  final String descriptionText;

  final IconData icon;

  final String? imageUrl;

  final OnLikeCallback onLike;

  final VoidCallback? onTap;

  final String? id;

  final bool isLiked;

  const _Card(this.text,

    {required this.descriptionText,

    this.icon = Icons.ac_unit_outlined,

    this.imageUrl,

    this.onLike,

    this.onTap,

    this.id,

    this.isLiked = false,

    });

  factory _Card.fromData(

    CardDate data, {

    OnLikeCallback onLike,

    VoidCallback? onTap,

    bool isLiked = false,

    }) =>

    _Card(

      data.text,

```

```

descriptionText: data.descriptionText,
icon: data.icon,
imageUrl: data.imageUrl,
onLike: onLike,
onTap: onTap,
id: data.id,
isLiked: isLiked,
);

@override
Widget build(BuildContext context) {
  return GestureDetector(
    onTap: onTap,
    child: Container(
      margin: const EdgeInsets.all(16),
      constraints: const BoxConstraints(minHeight:
140),
      decoration: BoxDecoration(
        color: Colors.white70,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(
          color: Colors.grey,
          width: 2,
        ),
        boxShadow: [
          BoxShadow(
            color: Colors.grey.withOpacity(.5),
            spreadRadius: 4,
            offset: const Offset(0, 5),
            blurRadius: 8,
            blurStyle: BlurStyle.normal)
        ],
      child: IntrinsicHeight(
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            ClipRRect(
              borderRadius: const BorderRadius.only(
                bottomLeft: Radius.circular(18),
                topLeft: Radius.circular(18),
              ),
              child: SizedBox(
                height: double.infinity,
                width: 120,
                child: Image.network(
                  imageUrl ?? "",
                  fit: BoxFit.cover,
                ),
              ),
            Expanded(
              child: Padding(
                padding: const EdgeInsets.only(left: 8.0),
                child: Column(

```

crossAxisAlignment:	onTap: () => onLike?.call(id, text,
CrossAxisAlignment.start,	isLiked),
children: [	child: AnimatedSwitcher(
Text(	duration: const Duration(milliseconds:
text,	300),
style:	child: isLiked
Theme.of(context).textTheme.headlineLarge,	? const Icon(
),	Icons.favorite,
Text(descriptionText, style:	color: Colors.redAccent,
Theme.of(context).textTheme.bodyLarge),	key: ValueKey<int>(0),
],	)
),	: const Icon(
),	Icons.favorite_border,
),	key: ValueKey<int>(1),
Column(	),
mainAxisAlignment:	),
MainAxisAlignment.spaceBetween,	),
children: [	),
Padding(	],
padding: const EdgeInsets.only(left: 8.0,	),
right: 8.0),	],
child: Icon(icon),	),
),	)),
Padding(	);
padding: const EdgeInsets.only(left: 8.0,	}
right: 8.0),	}
child: GestureDetector(	

```

import 'package:flutter_bloc/flutter_bloc.dart';
import
'package:flutter_labs_app/data/repositories/rick_rep
ository.dart';
import
'package:flutter_labs_app/presentation/home_page/
bloc/state.dart';
import 'events.dart';

class HomeBloc extends Bloc<HomeEvent,
HomeState> {
  final RickRepository rep;

  HomeBloc(this.rep) : super(const HomeState()) {
    on<HomeLoadDataEvent>(_onLoadData);
  }

  Future<void> _onLoadData(HomeLoadDataEvent
event, Emitter<HomeState> emit) async {
    if (event.nextPage == null) {
      emit(state.copyWith(isLoading: true));
    } else {
      emit(state.copyWith(isPaginationLoading: true));
    }

    String? error;

    final data = await rep.loadData(
      q: event.search,
      page: event.nextPage ?? 1,
      onError: (e) => error = e,
    );

    if (event.nextPage != null) {
      data?.data?.insertAll(0, state.data?.data ?? []);
    }

    emit(state.copyWith(
      isLoading: false,
      isPaginationLoading: false,
      data: data,
      error: error,
    ));
  }
}

abstract class HomeEvent {
  const HomeEvent();
}

class HomeLoadDataEvent extends HomeEvent {
  final String? search;

```



```

final int? nextPage;

const HomeLoadDataEvent({this.search,
this.nextPage});
}

```

```

import 'package:equatable/equatable.dart';

import
'package:flutter_labs_app/domain/models/home.dart' this.isPaginationLoading,
';
import '../../domain/models/card.dart';

```

```

class HomeState extends Equatable {

  final HomeData? data;

  final bool isLoading;

  final bool isPaginationLoading;

  final String? error;

  const HomeState({

    this.data,

    this.isLoading = false,

    this.isPaginationLoading = false,

    this.error,

  });

```

```

HomeState copyWith({

```

```

HomeData? data,

bool? isLoading,

bool? isPaginationLoading,

String? error,

}) =>

  HomeState(

    data: data ?? this.data,

    isLoading: isLoading ?? this.isLoading,

    isPaginationLoading: isPaginationLoading ??

    error: error ?? this.error,

  );

```

```

@override

List<Object?> get props => [

  data,

  isLoading,

  isPaginationLoading,

  error,

];
}

```

```

import 'package:flutter/material.dart';

import

'package:flutter_labs_app/domain/models/card.dart';

```

```

class DetailsPage extends StatelessWidget {
    final CardDate data;

    const DetailsPage(this.data, {super.key});

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(),
            body: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                    Padding(
                        padding: const EdgeInsets.only(bottom: 16.0),
                        child: Image.network(
                            data.imageUrl ?? "",
                            width: double.infinity,
                            fit: BoxFit.cover,
                        ),
                    ),
                    Padding(
                        padding: const EdgeInsets.only(bottom: 4.0),
                        child: Text(
                            data.text,
                            style:
Theme.of(context).textTheme.headlineLarge,
                        ),
                    ),
                ],
            ),
        );
    }
}

import 'package:flutter/widgets.dart';
import 'package:flutter_svg/flutter_svg.dart';
import '../components/resources.g.dart';

abstract class SvgObjects {
    static void init() {
        final pics = <String>[
            R.ASSETS_SVG_RU_SVG,
            R.ASSETS_SVG_US_SVG,
        ];
        for (final String p in pics) {
            final loader = SvgAssetLoader(p);
            svg.cache.putIfAbsent(loader.cacheKey(null), () =>
                loader.loadBytes(null));
        }
    }
}

```

<pre> } } } class SvgRu extends StatelessWidget { const SvgRu({super.key}); @override Widget build(BuildContext context) { return SvgPicture.asset(R.ASSETS_SVG_RU_SVG); } } class SvgUk extends StatelessWidget { const SvgUk({super.key}); @override Widget build(BuildContext context) { return SvgPicture.asset(R.ASSETS_SVG_US_SVG); } } import 'package:flutter_labs_app/domain/models/card.dart'; class HomeData { </pre>	<pre> final List<CardDate>? data; final int? nextPage; HomeData({this.data, this.nextPage}); } import 'package:flutter/material.dart'; class CardDate { final String text; final String descriptionText; final IconData icon; final String? imageUrl; final String? id; CardDate(this.text, { required this.descriptionText, this.icon = Icons.ac_unit_outlined, this.imageUrl = 'https://via.placeholder.com/150', this.id, }); } import 'package:dio/dio.dart'; </pre>
--	---

```
import @override

'package:flutter_labs_app/data/dtos/characters_dto. Future<HomeData?> loadData({
dart';      OnErrorCallback? onError,

import      String? q,

'package:flutter_labs_app/data/mappers/characters_ int page = 1,
mapper.dart'; int pageSize = 20,

import      }) async {

'package:flutter_labs_app/data/repositories/api_inter try {
face.dart';      const String url = '$_baseUrl/api/character';

import

'package:flutter_labs_app/domain/models/card.dart'; final Response<dynamic> response = await

import      _dio.get<Map<dynamic, dynamic>>(<

'package:flutter_labs_app/domain/models/home.dart url,

';      queryParameters: {

import      'name': q,

'package:pretty_dio_logger/pretty_dio_logger.dart'; 'page': page,

      },

class RickRepository extends ApiInterface {      });

static final Dio _dio = Dio()

      final CharactersDto dto =

      CharactersDto.fromJson(response.data as

      Map<String, dynamic>);

      final HomeData? data = dto.toDomain();

      return data;

static const String _baseUrl =      } on DioException catch (e) {

'https://rickandmortyapi.com';      onError?.call(e.error?.toString());

      return null;
}
```

```

    }

    }

    }

    'https://avatars.mds.yandex.net/i?id=7cb577fccf8b73
54b5248cb8101dd09433fa521f-4253662-images-
thumbs&n=13',

    ),

import 'package:flutter/material.dart';

import

'package:flutter_labs_app/data/repositories/api_inter
face.dart';

import

'package:flutter_labs_app/domain/models/card.dart';

import

'https://avatars.mds.yandex.net/i?id=34f57633c955c
47b56c68537076e5bfabb4a397b-4577841-images-
';

thumbs&n=13',

    ),

class MockRepository extends ApiInterface {

    @override

    Future<HomeData?> loadData({OnErrorCallback?

onError}) async {

        return HomeData(

            data: [

                CardDate(

                    'Hi',

                    descriptionText: 'hello',

                    icon: Icons.h_mobiledata_sharp,

                    imageUrl:

                    'https://avatars.mds.yandex.net/i?id=d3e8f5e5c373a
ec40e18fdb5bc28f98367e0f0d9-4271037-images-
thumbs&n=13',

                    ),

                CardDate(

                    'Last',

```

```

        descriptionText: 'hello',
      ),
    ],
  );
}

import
'package:flutter_labs_app/domain/models/card.dart';

import
'package:flutter_labs_app/domain/models/home.dart';

import
'package:flutter_labs_app/domain/models/card.dart';

import
'package:flutter_labs_app/domain/models/home.dart';

';

typedef OnErrorCallback = void Function(String?
error);

abstract class ApiInterface {
  Future<HomeData?> loadData({OnErrorCallback?
  onError});
}

import
'package:flutter_labs_app/data/dtos/characters_dto.
dart';

import
'package:flutter_labs_app/domain/models/card.dart';

import
'package:flutter_labs_app/domain/models/home.dart';

extension CharacterDataDtoModel on
CharacterDataDto {
  CardDate toDomain() => CardDate(
    name ?? 'UNKNOWN',
    imageUrl: image,
    descriptionText: _makeDescriptionText(species,
status),
    id: id.toString(),
  );

  String _makeDescriptionText(String? species, String?
status) {
    return species != null && status != null
      ? 'species: $species \n status: $status'
      : species != null
        ? 'species: $species'
        : status != null
          ? 'status: $status'
          : "";
  }
}

```

extension CharactersDataDtoModel on CharactersDto

```
{
    @JsonSerializable(createToJson: false)
    HomeData toDomain() => HomeData(
        data: results?.map((e) => e.toDomain()).toList(),
        nextPage: info?.next,
    );
}
```

```
import
'package:json_annotation/json_annotation.dart';
```

```
part 'characters_dto.g.dart';
```

```
@JsonSerializable(createToJson: false)
```

```
class CharactersDto {
```

```
    final List<CharacterDataDto>? results;
```

```
    final PaginationDto? info;
```

```
    const CharactersDto({
```

```
        this.results,
```

```
        this.info,
```

```
    });
```

```
    factory CharactersDto.fromJson(Map<String,
```

```
dynamic> json) => _$CharactersDtoFromJson(json);
```

```
}
```

```
class CharacterDataDto {
```

```
    final int? id;
```

```
    final String? name;
```

```
    final String? status;
```

```
    final String? species;
```

```
    final String? image;
```

```
    const CharacterDataDto({this.id, this.name,
        this.status, this.species, this.image});
```

```
    factory CharacterDataDto.fromJson(Map<String,
dynamic> json) =>
```

```
        _$CharacterDataDtoFromJson(json);
```

```
}
```

```
@JsonSerializable(createToJson: false)
```

```
class PaginationDto {
```

```
    final int? count;
```

```
    final int? pages;
```

```
    final int? next;
```

```
    const PaginationDto({this.count, this.pages,
        this.next});
```

```

factory PaginationDto.fromJson(Map<String, dynamic> json) {
    AppLocale get locale => AppLocale.of(this)!;

    return PaginationDto(
        count: (json['count'] as num?)?.toInt(),
        pages: (json['pages'] as num?)?.toInt(),
        next: _extractPageNumber(json['next'] as String?),
    );
}

```

```

static int? _extractPageNumber(String? url) {
    if (url == null) return null;

    final uri = Uri.tryParse(url);
    if (uri == null) return null;

    // Извлечение параметра "page" из строки
    запроса

    final page = uri.queryParameters['page'];
    return page != null ? int.tryParse(page) : null;
}
}

```

```

import 'package:flutter/widgets.dart';
import '../locale/l10n/app_locale.dart';

```

```

extension LocalContextX on BuildContext {

```